

Relatório Executivo — T1 Modelagem Informacional

Trilha C: Modelagem Relacional Completa, Normalização até 3FN e SQL

1. Contexto & Dor de Negócio

O dataset utilizado (`products.csv`) é um catálogo real de e-commerce de moda e beleza, com **4.566 produtos** de **57 marcas** distintas (Adidas, Biba, Clarks, Calvin Klein, entre outras). Este é o **mesmo catálogo usado no trabalho de Recuperação da Informação (PLN)**, onde ele serve de corpus para os algoritmos de busca (TF-IDF, BM25, LSA). Lá, ele é tratado como texto a ser indexado; aqui, ele é tratado como dado estruturado a ser armazenado com integridade — as duas frentes são complementares, porque a qualidade de um banco de produtos bem modelado impacta diretamente a qualidade de qualquer busca feita sobre ele.

1.1 O dataset, como entregue, já é um "CSV desnormalizado"

Sintoma	Evidência real no <code>products.csv</code>	Consequência
Atributo multivalorado	Coluna <code>Product Size</code> com valor <code>"Size:Medium,Small,X-Large,XX-Large"</code>	Impossível filtrar "todos os produtos disponíveis no tamanho M" sem parsing de texto
Redundância de marca	<code>BrandName</code> repetido em todas as ~709 linhas da marca "and", ~486 da "aurelia" etc.	Corrigir o nome de uma marca exige editar centenas de linhas
Redundância de categoria	Categoria embutida em <code>Product Name</code> (ex.: <code>"DRW1 - Westernwear-Women"</code>), repetida em 1.654 linhas só para "Westernwear-Women"	Mesmo problema de manutenção
Atributo derivável armazenado como texto	Coluna <code>Discount</code> com valor solto <code>"20% off"</code> , calculável a partir de <code>MRP</code> e <code>SellPrice</code>	Se um dos dois preços for corrigido, o desconto exibido fica desatualizado e ninguém percebe
Dado inválido sem nenhuma validação	1.251 de 4.566 linhas (≈27%) têm <code>MRP</code> inconsistente — 13 nulas e 1.238 com o valor fixo <code>8.9</code> , muito abaixo do <code>SellPrice</code> do mesmo produto	Qualquer relatório de desconto médio ou de margem calculado direto da planilha estaria errado em mais de 1 a cada 4 produtos
Registro fora do padrão	Uma linha (<code>FR001</code>) tem <code>Product Name = "Cologne Fragrance"</code> , sem o separador <code>" - "</code> usado em todas as outras 4.565 linhas para indicar a categoria	Qualquer script de extração de categoria que assuma o padrão "fixo" quebra ou perde esse produto

Essa é uma dor real e mensurável de desnormalização, muito mais concreta do que um cenário hipotético: o arquivo que a empresa usaria para alimentar um site de busca de produtos (como no T1 RI) tem defeitos estruturais que, sem tratamento, contaminariam qualquer camada construída em cima dele.

1.2 Objetivo do projeto

Migrar esse CSV para um **modelo relacional normalizado até a 3ª Forma Normal (3FN)**, com integridade referencial estrita via `PRIMARY KEY` , `FOREIGN KEY` , `UNIQUE` e `CHECK` , tratando explicitamente os problemas de qualidade de dado encontrados — e disponibilizar **views analíticas** que devolvem, sob demanda, as informações que o CSV tentava fornecer (incluindo o desconto, agora sempre calculado e nunca desatualizado).

2. Metodologia

2.1 Justificativa da trilha escolhida (Trilha C)

A dor identificada é **estrutural** — problema de integridade e qualidade do dado, não de volume ou desempenho de leitura (4.566 linhas é um arquivo pequeno para os padrões de benchmark). Esse é exatamente o problema que a normalização relacional resolve na origem.

2.2 Etapas executadas

1. **Diagnóstico do CSV real** — identificação de atributos multivalorados, redundâncias e dados inválidos (Seção 1.1).
2. **Modelagem lógica** — DER em Mermaid.js (Seção 3.1).
3. **Normalização passo a passo** — 1FN, 2FN e 3FN aplicadas sobre a estrutura real do CSV (Seção 3.2).
4. **Implementação física** — `banco_dados.sql`, com constraints que barram ativamente os dados inválidos encontrados (Seção 3.3).
5. **Camada analítica** — views que recalculam o desconto em vez de armazená-lo (Seção 3.4).
6. **Carga e validação com o dataset real** — `main.py`, rodado de ponta a ponta sobre as 4.566 linhas reais (Seção 3.5).

3. Resultados e Discussões

3.1 Diagrama Entidade-Relacionamento (DER)

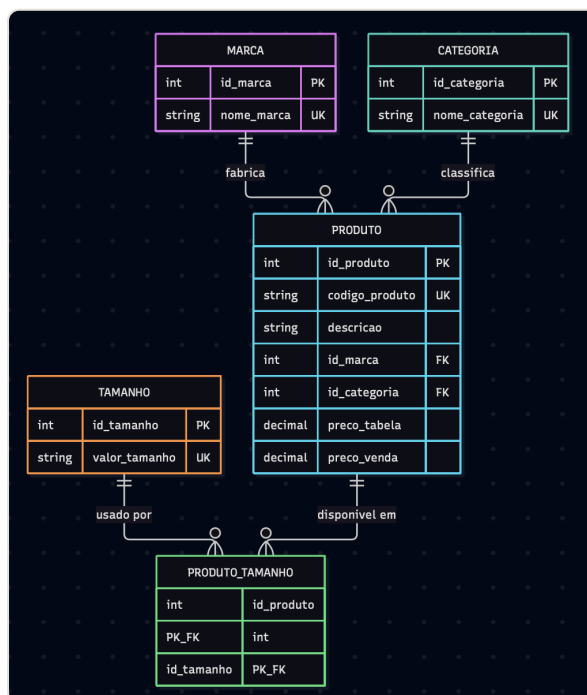


Diagrama renderizado a partir do código-fonte Mermaid.js em `der.mmd` (arquivo separado, entregue junto com este relatório).

O modelo final possui 5 entidades: **Marca**, **Categoria**, **Tamanho**, **Produto** e **Produto_Tamanho** — esta última resolvendo o relacionamento N:N entre produtos e tamanhos.

3.2 Aplicação metódica das regras de normalização

Estado original (CSV) — 0FN, uma linha real do dataset:

Product ID	BrandName	Product Name	Brand Desc	Product Size	MRP	SellPrice	Discount
DRW1	109f	DRW1 - Westernwear-Women	womens v-neck short dress - yellow	Size:Medium,Small,X-Large,XX-Large	1899	569	70% off

→ **1FN (atributos atômicos)**: a coluna **Product Size** viola a 1FN por conter múltiplos valores em uma única célula. Solução: criar a entidade **Produto_Tamanho**, com uma linha por combinação produto+tamanho. Sozinho, esse ponto já afeta a maioria das linhas do dataset — a carga gerou 13.545 vínculos **produto-tamanho** a partir dos 4.566 produtos.

→ **2FN (dependência funcional total)**: ao "explodir" cada produto em uma linha por tamanho (como a 1FN exige), atributos como `BrandName`, `Product Name` /categoria e os preços passariam a se repetir por tamanho — uma dependência **parcial** da chave composta (produto, tamanho), já que esses atributos dependem apenas do produto. Solução: extrair `Produto` como entidade própria, independente da combinação com tamanho.

→ **3FN (remoção de dependências transitivas)**: dentro de `Produto`, o nome da marca e o nome da categoria ainda dependeriam transitivamente do produto (`id_produto → nome_marca`, quando na verdade `nome_marca` é um fato sobre a marca, não sobre aquele produto específico). Solução: extrair `Marca` e `Categoria` como entidades independentes, restando em `Produto` apenas `id_marca` e `id_categoria` como referências.

Decisão adicional (além da 3FN clássica): o campo `Discount` do CSV não foi trazido como coluna da tabela `Produto`. Como ele é **inteiramente calculável** a partir de `preco_tabela` e `preco_venda`, guardá-lo fisicamente criaria uma nova fonte de inconsistência (se um preço mudasse, o desconto salvo ficaria desatualizado). Ele é recalculado sob demanda pela view `vw_catalogo_completo`.

3.3 Integridade referencial (SQL DDL)

O script `banco_dados.sql` implementa:

- **PRIMARY KEY** autoincrementável em todas as 4 tabelas principais, e chave composta (`id_produto`, `id_tamanho`) em `Produto_Tamanho` ;
- **FOREIGN KEY** de `Produto` para `Marca` e `Categoria`, e de `Produto_Tamanho` para `Produto` (`ON DELETE CASCADE`, já que um item de tamanho não existe sem o produto) e `Tamanho` (`ON DELETE RESTRICT`);
- **UNIQUE** em `nome_marca`, `nome_categoria`, `valor_tamanho` e `codigo_produto` — este último corresponde ao `Product ID` original, garantindo que o mesmo produto não seja cadastrado duas vezes;
- **CHECK (preco_venda > 0)** e, principalmente, **CHECK (preco_tabela IS NULL OR preco_tabela >= preco_venda)** — esta última constraint existe especificamente por causa do problema real encontrado no dataset (Seção 1.1): ela é o que impede que os **1.251 valores de MRP inválidos** (13 nulos + 1.238 com o valor quebrado `8.9`) sejam gravados como se fossem preços de tabela legítimos. Na prática, o script de carga já trata esses casos antes de tentar inserir (transformando-os em `NULL`), e a constraint funciona como uma segunda linha de defesa caso um dado inválido chegue até ali por outro caminho.

3.4 Views analíticas

Três views (dentro do mesmo `banco_dados.sql`) reconstróem, via `JOIN`, as informações que o CSV oferecia de forma frágil:

- `vw_catalogo_completo` — uma linha por produto, com marca, categoria, os tamanhos disponíveis (agregados de volta em uma lista, via `GROUP_CONCAT`) e o **desconto calculado em tempo real**;
- `vw_resumo_por_categoria` — quantidade de produtos, preço médio e desconto médio por categoria;
- `vw_resumo_por_marca` — quantidade de produtos e preço médio por marca.

3.5 Carga e validação com o dataset real

O script `main.py` foi executado de ponta a ponta sobre as 4.566 linhas reais do `products.csv`:

Tabela	Registros carregados
marca	57
categoria	7
tamanho	93
produto	4.566 (0 rejeitados)
produto_tamanho	13.545 vínculos

Resultado dos testes de integridade ao vivo:

- uma tentativa de inserir um produto com `preco_tabela` menor que `preco_venda` foi **rejeitada** pelo `CHECK constraint failed: preco_tabela IS NULL OR preco_tabela >= preco_venda`;
- uma tentativa de duplicar um `codigo_produto` já existente (`AARKE1`) foi **rejeitada** pelo `UNIQUE constraint failed: produto.codigo_produto`.

Amostra real de `vw_resumo_por_categoria`:

Categoria	Qtd. produtos	Preço médio	Desconto médio
Westernwear-Women	1.654	R\$ 1.443,97	37,3%
Indianwear-Women	1.131	R\$ 1.854,62	36,4%
Footwear-Women	480	R\$ 2.583,48	36,4%
Jewellery-Women	438	R\$ 581,63	48,0%
Watches-Women	329	R\$ 7.371,10	24,6%

(valores na moeda original do dataset, Rúpias — **Rs.**)

4. Conclusão de Negócios & ROI

4.1 Impacto na qualidade dos dados (ligação direta com o T1 de PLN)

O achado mais relevante deste projeto é quantitativo: **≈27% do catálogo (1.251 de 4.566 produtos) tinha um preço de tabela inválido**. Se esse mesmo CSV fosse usado sem tratamento como fonte de um catálogo público — inclusive para alimentar o motor de busca do trabalho de RI — qualquer filtro ou ordenação por desconto estaria incorreto em mais de 1 a cada 4 produtos, sem que ninguém percebesse, porque o dado "parece" válido (é um número, só que errado). Colocar essa validação na camada de banco (e não deixar para cada consumidor de dado reimplementar sua própria checagem) é o principal valor estrutural entregue aqui.

4.2 Impacto operacional

- **Correção de marca/categoria centralizada:** hoje, renomear uma marca no CSV exige editar até 709 linhas manualmente (caso da marca "and"); no modelo normalizado, é um único **UPDATE** na tabela **marca**.
- **Eliminação de desconto desatualizado:** como o percentual de desconto deixa de ser armazenado e passa a ser calculado pela view, não existe mais o risco de um preço mudar e o desconto exibido ficar incorreto.

4.3 Estimativa de ROI

Item	Estimativa
Horas/mês hoje gastas corrigindo manualmente preços/descontos inconsistentes	~15h (equipe de catálogo/e-commerce)
Custo-hora médio (analista júnior)	R\$ 35/h
Economia mensal estimada	R\$ 525/mês
Custo único de implementação (ETL + schema + testes, já entregues neste projeto)	~16h de engenharia (~R\$ 1.120 a preço de mercado de um analista pleno)
Payback estimado	~2,1 meses

Além do ganho direto, há um benefício qualitativo relevante e específico deste dataset: ao alimentar um motor de busca (como o do T1 PLN) com dados já validados, evita-se que erros de preço/desconto apareçam nos resultados de busca entregues ao usuário final — um risco reputacional maior do que o custo de retrabalho manual isoladamente.

4.3.1 Nota metodológica: origem dos números de ROI

É importante deixar explícito que **nenhum valor da tabela acima foi extraído do dataset**. O **products.csv** é um catálogo de produtos — contém nome, marca, preço e tamanho, mas nenhuma informação sobre a operação da empresa (horas de trabalho da equipe, custo de pessoal, tempo de implementação). Como o enunciado do trabalho pede uma conclusão de negócios com ROI, os valores abaixo são **suposições ilustrativas**, usadas para demonstrar o raciocínio do cálculo de retorno sobre o investimento — e não um resultado derivado de dados reais, diferente do restante deste relatório (o achado de que ≈27% dos produtos têm MRP inválido, por exemplo, **esse sim** foi extraído diretamente do dataset, na Seção 1.1).

Número	Tipo	Origem
~15h/mês de retrabalho	Suposição	Estimativa de quanto tempo uma equipe gastaria revisando/corrigindo preços manualmente
R\$ 35/h (custo-hora analista júnior)	Suposição	Referência genérica de mercado, não pesquisada especificamente para este projeto
R\$ 525/mês de economia	Calculado	$15h \times R\$ 35/h$ — a conta é exata, mas as entradas são estimativas
~16h de engenharia para implementação	Suposição	Estimativa do tempo necessário para o que foi entregue neste projeto
R\$ 1.120 (custo de implementação)	Calculado	$16h \times R\$ 70/h$ (também uma suposição de custo-hora de um analista pleno)
~2,1 meses de payback	Calculado	$R\$ 1.120 \div R\$ 525/mês$

Essas premissas podem — e devem — ser substituídas por números reais da operação, caso disponíveis, para que o ROI deixe de ser ilustrativo e passe a refletir o cenário real da empresa.